

Cartographie du projet `docs_parser_15`

Pipeline de **traduction de documents PDF avec préservation de la mise en page (WYSIWYG)** : on prend un PDF source, on comprend chaque page, on traduit le texte, puis on **recompose** la page traduite à l'identique (même fonds, mêmes figures, même typographie).

Architecture en une ligne

```
SOURCE (PDF / image / Office)
  → [moteur historique : ocr_server / translator / reconstructor]
(l'ancien, "legacy")
  → [pipeline moderne]  PAGEPRINT → PAGETRANSLATE → PAGERECONSTRUCT →
PUBREADY (QA)
```

Le projet contient **deux générations** qui coexistent :

1. **Le moteur historique** (fichiers à la racine) : monolithique, tout passe par `ocr_server.py`.
2. **Le pipeline moderne** (`pipelines/`, `pageprint/`, `pagetranslate/`, `pagereconstruct/`, `pubready/`) : un « clone propre » du moteur historique, découpé en unités testables, chacune avec un contrat de données explicite.

Le module que tu debugges (`pagereconstruct/`) est la **3^e tête** : c'est lui qui efface le texte source et redessine le texte traduit. C'est aussi la section la plus détaillée ci-dessous.

1. Le moteur historique (fichiers à la racine)

Ce sont les gros fichiers monolithiques, base de tout le savoir métier. Le pipeline moderne les « porte » progressivement.

Fichier	Rôle
<code>ocr_server.py</code> (56361.)	Serveur FastAPI + orchestrateur historique. Reçoit le PDF, convertit l'Office en PDF (LibreOffice headless), rend les pages, lance OCR, extraction, traduction et reconstruction. C'est le « cerveau » de l'ancienne génération que <code>pipelines/orchestrator.py</code> cherche à remplacer proprement.
<code>translator.py</code> (73981.)	DocumentTranslator : le traducteur métier complet (sélection sémantique, contexte page/document, terminologie, ton, protection des tokens non traduisibles, contrôle de longueur/overflow). Tout le reste (<code>pagetranslate</code> , <code>translation_engines</code>) gravite autour de lui.
<code>reconstructor.py</code> (86451.)	DocumentReconstructor : le reconstituteur historique. Géométrie des blocs, gabarits de ligne, placement, expansion de bbox, gestion des overlays. <code>pagereconstruct/</code> est la réécriture moderne et contractuelle de ce fichier.
<code>native_pdf_extractor.py</code>	Extraction native du PDF via PyMuPDF : texte, polices, couleurs, images, dessins vectoriels, zones non textuelles (bbox en points).
<code>structure_extractor.py</code>	VisualAttributeExtractor + DocumentParser + LayoutV2Builder : analyse visuelle d'une page et construction de la structure logique (blocs > lignes > phrases > spans).
<code>perfect_document_extractor.py</code>	Extracteur expérimental « haute fidélité », autonome, qui produit un modèle documentaire plus riche que le pipeline courant (ne le modifie pas).

Fichier	Rôle
<code>perfect_extraction_to_reconstruction.py</code>	Adaptateur expérimental : convertit la sortie du <code>perfect_document_extractor</code> vers l'entrée du <code>DocumentReconstructor</code> .
<code>perfect_reconstructor.py</code>	Sous-classe expérimentale du restructeur, change seulement le dispatch pour les pages produites par l'extracteur « parfait ».
<code>final_page_compiler.py</code> (14111.)	<code>ContinuousFinalPageCompiler</code> : compile une page finale continue (items texte, formules, opérations de dessin).

Compréhension / classification de page

Fichier	Rôle
<code>layout_descriptor.py</code> (24801.)	<code>LayoutDescriptorBuilder</code> : décrit la mise en page d'une page (colonnes, zones, structure). Version historique.
<code>layout_descriptor_v3.py</code> (14761.)	Version 3 du descripteur de layout.
<code>layout_compiler.py</code>	Compile des placements de layout (<code>LayoutPlacement</code>), calculs de chevauchement/colonnes.
<code>layout_optimizer.py</code>	<code>LayoutOptimizer</code> : optimisation de placement.
<code>layout_ai_enricher.py</code> (6461.)	Enrichissement de la mise en page par modèle IA visuel (<code>LayoutAIEnricher</code>).
<code>visual_document_layout_model.py</code>	<code>VisualDocumentLayoutModel</code> : modèle de layout visuel (détection de structure par vision).
<code>page_case_classifier.py</code> (8541.)	<code>PageCaseClassifier</code> : classe le « cas » d'une page (couverture, table des matières, index, corps, figure...).
<code>page_case_classifier_v2.py</code>	Version 2 du classifieur de cas de page.
<code>page_family_registry.py</code>	Registre des « familles » de pages et de leurs configs.
<code>page_profile_registry.py</code>	Registre des profils de page.

Fichier	Rôle
<code>page_policy_matrix.py</code> (5101.)	<code>PagePolicyMatrix</code> : matrice de politiques (que faire selon le type de page/bloc).
<code>context_classifier.py</code>	<code>ContextClassifier</code> : classe le contexte (domaine, type de contenu).
<code>block_typology.py</code>	<code>classify_block_typology</code> : typologie d'un bloc (titre, paragraphe, liste...).
<code>special_region_detector.py</code> (1175 l.)	Détecte les zones spéciales (formules, tables, code, images, logos) à protéger.

Relations, géométrie, règles

Fichier	Rôle
<code>element_relations.py</code>	Enrichit les relations entre éléments (ordre de lecture, paires, confiance).
<code>element_relations_ai.py</code>	<code>ElementRelationsAIEnricher</code> : même chose, version IA.
<code>element_rulesets.py</code> (8431.)	Construit des jeux de règles par phrase/bloc (annotation TOC, etc.).
<code>relative_geometry.py</code>	Enrichit la géométrie relative de la page (features, taille papier, direction de lecture).
<code>positioning_policy.py</code>	Politique de positionnement par bloc (scores sémantiques, alignement).
<code>logical_block_rebuilder.py</code>	Reconstruit les blocs logiques à partir de la géométrie.
<code>coverage_validator.py</code> (8331.)	Valideur de couverture : vérifie que tout le texte source est bien couvert (traduit/préservé/exclu) dans le rendu. Pièce centrale de la garantie « aucun texte perdu ».
<code>page_extraction_postprocessors.py</code> (12421.)	Post-traitements après extraction (fusion de bbox, tri des lignes, nettoyage).

Texte, style, traduction (historique)

Fichier	Rôle
<code>text_composer.py</code>	<code>TextComposer</code> : composition de texte (wrap, options de mise en ligne).
<code>style_profiler.py</code>	Profilage de style (couleurs, contraste, luminance).
<code>style_tone_classifier.py</code>	<code>StyleToneClassifier</code> : classe le ton/style du document.
<code>font_resolver.py</code>	<code>FontResolver</code> : résolution des polices (polices embarquées, répertoires système).
<code>terminology_manager.py</code>	<code>TerminologyManager</code> : gestion de la terminologie/glossaire.
<code>translation_memory.py</code>	<code>TranslationMemory</code> (historique).
<code>translation_validator.py</code>	<code>TranslationValidator</code> : validation de traduction.
<code>llm_semantic_corrector.py</code> (14111.)	Post-processeur LLM léger : corrige les segmentations sémantiques ambiguës (titres implicites, regroupement de légendes...).

Suppression de texte / fonds

Fichier	Rôle
<code>background_inpainter.py</code>	<code>BackgroundInpainter</code> : efface le texte source de l'image pour produire un fond propre (inpainting).
<code>remove_text_generic.py</code>	Suppression de texte générique (masque EAST + inpainting OpenCV).
<code>remove_text_dbnet_opencv.py</code>	Suppression de texte via DBNet + OpenCV (modèle de détection de texte).
<code>text_removal_strategy.py</code>	<code>TextRemovalStrategy</code> : choisit la stratégie de suppression.

Sortie / QA / divers racine

Fichier	Rôle
<code>html_exporter.py</code>	<code>HtmlStyleExporter</code> : export HTML stylé du document.
<code>visual_compare.py</code>	Comparaison visuelle (signatures de hauteur/gap/alignement) original vs reconstruit.
<code>publication_qa.py</code> (736 l.)	QA publication (version historique, comparaison à la source).
<code>render_quality_feedback.py</code>	Construit un rapport de 2 ^e passe sur les défauts de rendu.
<code>evaluate_layout_quality.py</code>	Évalue la qualité de layout (ratio de chevauchement).
<code>runtime_config.py</code>	Configure l'environnement « agentless » (variables d'env par défaut). Importé en tout premier par <code>translator.py</code> .

2. `pipelines/` — l'orchestrateur moderne (clone propre d'`ocr_server.py`)

Chaîne : `SourceLoader` → `PageRenderer` → `RawExtractors` → `PageUnderstanding` → `PAGEPRINT`.

Fichier	Rôle
<code>pipelines/__init__.py</code>	Décrit l'architecture cible de l'orchestrateur.
<code>pipelines/orchestrator.py</code>	PipelineOrchestrator : enchaîne les unités page par page et produit l' <code>INPUT_DATA</code> canonique. C'est le remplaçant propre du monolithe <code>ocr_server.py</code> .
<code>pipelines/source_loader.py</code>	Unité 1 : lit PDF/image/Office, convertit l'Office en PDF, calcule le hash et les métadonnées source.
<code>pipelines/page_renderer.py</code>	Unité 2 : rend la page en image (DPI, scale px ↔ pt). Le pixel ne sert qu'à l'OCR/masques/debug ; le modèle reste en points.
<code>pipelines/raw_extractors.py</code>	Unité 3 : extraction brute multi-sources (NativePDF + OCR optionnel).
<code>pipelines/page_understanding.py</code>	Unité 4 : compréhension de page (rôle, famille, type de doc, style, relations) via <code>LayoutV2Builder</code> , avant PAGEPRINT.
<code>pipelines/pageprint_stage.py</code>	Unité 5 : lance la tête PAGEPRINT (normalise → résout conflits → units → regions → graph → polices → constraints → quality).
<code>pipelines/ocr_router.py</code>	Décide si l'OCR tourne en pleine page, sur des régions visuelles, ou pas du tout.
<code>pipelines/background_cleaner.py</code>	Génère le fond « text-removed » (inpainting du texte source).
<code>pipelines/background_cover.py</code>	Fond déterministe « couvre-texte » : chaque ligne source à redessiner est recouverte d'une couleur estimée localement.
<code>pipelines/document_context.py</code>	Contexte document : en-têtes/pieds de page répétés, marques.

3. `pageprint/` — PAGEPRINT (1^{re} tête : compréhension canonique)

Transforme une page source en `INPUT_DATA` : représentation **canonique, normalisée, enrichie, vérifiable**, consommée par tous les modules aval. C'est la **source de vérité**.

Cœur

Fichier	Rôle
<code>pageprint/__init__.py</code>	Description de la tête PAGEPRINT.
<code>pageprint/builder.py</code> (1029 l.)	Construit l' <code>INPUT_DATA</code> (l'empreinte canonique de la page). Orchestrateur de tous les sous-modules ci-dessous.
<code>pageprint/schema.py</code>	Contrat canonique <code>pageprint.input.v1</code> (DTOs, constantes).
<code>pageprint/normalizer.py</code>	Normalise unités/points/styles. Règle d'or : modèle interne en points , pixels seulement aux frontières image/OCR.
<code>pageprint/unit_factory.py</code> (857 l.)	Transforme blocs/lignes/phrases/spans en units canoniques (liste plate, chaque unité porte ses champs requis).
<code>pageprint/region_index.py</code>	Indexe régions/zones/objets (images, dessins, tables, formules, code...) et calcule les appartenances.
<code>pageprint/graph_builder.py</code>	Construit le graphe documentaire (page $\supset$ region $\supset$ block $\supset$ line $\supset$ phrase $\supset$ span).
<code>pageprint/graph_query.py</code>	Helpers de requête graphe/rôle, partagés avec le fallback PAGETRANSLATE.
<code>pageprint/role_resolver.py</code> (508 l.)	Résout le rôle documentaire de chaque unité (titre, corps, légende, formule...).
<code>pageprint/semantic_builder.py</code> (516 l.)	Construit les unités sémantiques à partir des rôles et structures logiques.

Décision, preuve, qualité

Fichier	Rôle
<code>pageprint/evidence_resolver.py</code>	Résout les conflits entre sources (native_pdf / ocr / layout_ai / detector / heuristics / LLM) : qui gagne.
<code>pageprint/evidence/__init__.py</code>	Sous-paquet preuve.
<code>pageprint/evidence/claim_model.py</code>	DTO de « claim » (revendication d'une source).

Fichier	Rôle
<code>pageprint/evidence/collector.py</code>	Collecte les revendications concurrentes depuis les unités normalisées.
<code>pageprint/evidence/resolver.py</code>	Résout ces revendications en compréhension d'unité.
<code>pageprint/provenance.py</code>	Mémoire de décision : pourquoi tel bloc est une légende, pourquoi telle phrase n'est pas traduite... (pipeline rejouable).
<code>pageprint/quality_assessor.py</code>	Score de confiance, risques, fragments suspects, unités faibles, risques aval .
<code>pageprint/policy_compiler.py</code>	Compile les policies (ordres exécutables aval : <code>skip_translation</code> , <code>background_only</code> ...).
<code>pageprint/preservation_compiler.py</code>	Compile les modes de préservation depuis rôles/preuve.
<code>pageprint/constraint_compiler.py</code>	Compile les contraintes WYSIWYG par unité (<code>preserve_bbox</code> , <code>allow_reflow</code> , <code>allow_wrap</code> , <code>preserve_alignment</code> ...).
<code>pageprint/view_compiler.py</code>	Compile les plans d'exécution aval (les « vues » consommées par <code>pagetranslate</code> / <code>pagereconstruct</code>).
<code>pageprint/validators.py</code>	Valide que l' <code>INPUT_DATA</code> est exploitable (<code>bbox</code> en points, <code>unit_id</code> uniques, parents existants...).
<code>pageprint/functional_validator.py</code>	Validation fonctionnelle au-delà du schéma.
<code>pageprint/serializers.py</code>	Sérialisation JSON de l' <code>INPUT_DATA</code> .
<code>pageprint/text_postprocessors.py</code>	Nettoyage pré-translation (dé-césure intra-texte et inter-segments).

Détection de régions

Fichier	Rôle
<code>pageprint/detection/__init__.py</code>	Phase interne de détection de régions.
<code>pageprint/detection/builder.py</code>	Orchestre la détection (détecteur hybride legacy + fusion avec régions amont).
<code>pageprint/detection/schema.py</code>	Constantes de schéma de détection.

Constructeurs de structures logiques (`pageprint/structure_builders/`)

Chacun construit un type d'unité logique sans dupliquer bloc/ligne/phrased :

Fichier	Construit...
<code>__init__.py</code> / <code>common.py</code>	Paquet + helpers partagés.
<code>heading_builder.py</code>	Les titres.
<code>body_paragraph_builder.py</code>	Les paragraphes de corps.
<code>list_builder.py</code>	Les items de liste.
<code>table_builder.py</code> (243l.)	Les tables (détection de lignes/grille).
<code>toc_builder.py</code>	Les entrées de table des matières.
<code>index_builder.py</code>	Les entrées d'index.
<code>caption_builder.py</code>	Les légendes.
<code>figure_builder.py</code>	Les figures (+ légendes proches, labels de diagramme).
<code>formula_builder.py</code>	Les unités de formule.
<code>code_builder.py</code>	Les blocs de code.
<code>author_bio_builder.py</code>	Les bios d'auteur.
<code>publisher_mark_builder.py</code>	Les marques éditeur / artefacts (à excludre de la traduction).

4. `pagetranslate/` — PAGETRANSLATE (2^e tête : traduction)

Consomme l'`INPUT_DATA` de PAGEPRINT, sélectionne les unités traduisibles, ajoute le contexte, traduit, puis réinjecte. Produit `translated_input_data`.

Fichier	Rôle
<code>pagetranslate/__init__.py</code> / <code>README.md</code>	Description de la tête.
<code>pagetranslate/builder.py</code> (486l.)	Pipeline métier : sélection → protection → traduction → contrôle qualité → projection.
<code>pagetranslate/schema.py</code>	Constantes et DTOs du contrat de traduction.
<code>pagetranslate/selector.py</code> (480l.)	Sélection sémantique des unités traduisibles (<code>semantic_phrase > group > phrase > line > block</code>), exclut <code>word/char</code> .
<code>pagetranslate/coalescer.py</code>	Fusionne les unités visuelles en phrases sémantiques quand nécessaire.
<code>pagetranslate/sentence_boundary.py</code>	Frontières de phrase (début/fin, abréviations, multi-ligne, hard break vs soft wrap).
<code>pagetranslate/protection.py</code>	Placeholderise puis restaure les tokens intouchables (URL, DOI, emails, nombres, unités, formules, chemins, références).
<code>pagetranslate/technical_protection.py</code>	Protège les tokens techniques dans code/table (<code>None/True/False</code> , <code>Conv2D</code> , shapes, SQL, chemins...).
<code>pagetranslate/context_builder.py</code>	Construit le contexte (avant/après, page, domaine, style, ton, contraintes WYSIWYG).
<code>pagetranslate/terminology.py</code>	Support terminologie/glossaire.
<code>pagetranslate/translator_bridge.py</code>	Appel propre à <code>DocumentTranslator</code> , avec retry si sortie vide/identique.
<code>pagetranslate/quality.py</code> (239l.)	Contrôles qualité + <code>needs_review</code> (expansion, tokens protégés, nombres, unités).
<code>pagetranslate/functional_validator.py</code>	Vérifications fonctionnelles de la sortie.
<code>pagetranslate/projection.py</code> (399l.)	Réinjecte les traductions dans l' <code>INPUT_DATA</code> + vue compatible

Fichier	Rôle
	reconstruction.
<code>pagetranslate/text_survival.py</code> (496l.)	Invariant dur : toute ligne source visible DOIT avoir un chemin de sortie (traduite OU préservée OU exclue avec raison). Privilégie la présence du texte à la beauté du layout.
<code>pagetranslate/translation_plan_reader.py</code>	Lit <code>views.translation_plan</code> de PAGEPRINT.
<code>pagetranslate/text_utils.py</code>	Petits helpers texte partagés.

5. `pagereconstruct/` — PAGERECONSTRUCT (3^e tête : recomposition) ★

« Not a text drawer — a document recomposition solver. » On ne redessine pas du texte : on respecte un plan de couches, protections, patches, styles et contraintes. C'est le **module que tu debugges**. Les deux bugs connus (texte source non effacé + bbox qui écrase le texte sur la 1^{re} ligne) vivent ici, surtout dans `layout_box_resolver.py`, `background_resolver.py`, `patch_planner.py` et les backends.

Pipeline interne

```
translated_input_data
  → input_adapter          (4 vues)
  → plan_compiler          (fusion, anti double-rendu)
    layout_box_resolver / style_resolver / background_resolver /
patch_planner
  → PageRenderPlan
  → renderer_dispatcher + renderers/
  → backends/ (pdf_vector | raster_debug)
  → validator / quality / visual_qa → status ok|review|ko
```

Entrée & compilation du plan

Fichier	Rôle
<code>pagereconstruct/__init__.py</code> / <code>README.md</code>	Description + philosophie de la tête.
<code>pagereconstruct/schema.py</code>	Contrats de données : le <code>PageRenderPlan</code> intermédiaire (couches + protections + ids

Fichier	Rôle
	consumed/excluded).
<code>pagereconstruct/input_adapter.py</code>	Normalise <code>translated_input_data</code> en 4 plans : reconstruction_units (texte traduit), reconstruction_plan, preservation_plan, exclusion_plan.
<code>pagereconstruct/plan_compiler.py</code> (7791.)	Compile le <code>PageRenderPlan</code> : fusionne les 4 vues + géométrie + couches visuelles, calcule consumed/excluded (anti double-rendu). Pièce maîtresse.

Résolveurs (où vivent tes bugs) ●

Fichier	Rôle
<code>pagereconstruct/layout_box_resolver.py</code>	● Résout layout/patch/anchor bbox par rôle. Corrige précisément le bug que tu décris : un paragraphe multi-ligne est effacé sur toute sa surface mais le texte est redessiné dans la bbox de la 1^{re} ligne . Ici, le texte de flux doit se mettre en page dans <code>layout_bbox</code> = bloc logique complet. À regarder en priorité.
<code>pagereconstruct/background_resolver.py</code>	● Décide le fond + le risque de fuite du texte source : <code>clean_background > source_background (debug) > blank_degraded</code> . Si le texte source n'est pas effacé, c'est souvent ici ou dans le patch_planner.
<code>pagereconstruct/patch_planner.py</code>	● Planifie les zones de patch (cleanup) au-dessus du texte source. Chaque unité traduite reçoit un patch sur sa zone source ; refusé sur région protégée.
<code>pagereconstruct/style_resolver.py</code>	Résout un style typographique exploitable pour CHAQUE unité (aucune unité ne doit arriver au backend avec <code>style = {}</code>).
<code>pagereconstruct/font_resolver_bridge.py</code>	Déduit la vraie classe de police depuis les noms de subset PDF mutilés (serif/sans/mono).

Fichier	Rôle
<code>pagereconstruct/font_size_sanitizer.py</code>	● Répare les tailles de police absurdes (le démo montrait ~4.78pt pour du corps qui fait 9-11pt — l'extracteur sort parfois une métrique de glyphe, pas la taille réelle). Lié à ton 2 ^e bug (texte illisible).

Contrats

Fichier	Rôle
<code>pagereconstruct/final_contract.py</code>	FinalReconstructionContract : source unique et non ambiguë du rendu (fusion pageprint + pagetranslate + savoir legacy).
<code>pagereconstruct/block_contract.py</code>	Contrat complet d'un bloc à rendre (géométrie + style + politique + préservation + qualité).
<code>pagereconstruct/layout_contract.py</code>	Géométrie + libertés de déplacement (gabarits de ligne, padding, keep_with).
<code>pagereconstruct/style_contract.py</code>	Typographie figée d'une unité (jamais inventée par le rendu ; provenance extracted/inferred/repairé).
<code>pagereconstruct/background_contract.py</code>	Quel fond + risque de fuite (fond non nettoyé = bloquant en publication).
<code>pagereconstruct/preservation_contract.py</code>	Objets gardés tels quels (pixels) + texte exact (overlays/underlays).
<code>pagereconstruct/object_contract.py</code>	Identité/politique d'un objet de page.
<code>pagereconstruct/quality_contract.py</code>	Seuils publication-ready (gates durs : must_render / no clip / no overlap).
<code>pagereconstruct/page_level_contracts.py</code>	Contrats des objets « page-level » (politique de traduction/préservation/ancrage explicite).

Composition (avant rendu)

Fichier	Rôle
<code>pagereconstruct/composition/block_planner.py</code>	Place les GRANDS blocs avant l'intra-bloc (solveur multi-blocs).
<code>pagereconstruct/composition/intrablock_composer.py</code>	Compose le contenu DANS un bloc (lignes, runs, objets inline — le renderer ne compose plus).
<code>pagereconstruct/composition/typography_planner.py</code>	Produit le <code>TypographyPlan</code> (échelle de page, em estimé) avant le rendu.
<code>pagereconstruct/composition/special_zone_preserver.py</code>	Classe les objets non traduisibles en 4 niveaux (block/inline/page/background).

Solveurs de placement

Fichier	Rôle
<code>pagereconstruct/candidate_engine.py</code>	Génère et score des candidats de rendu (6 scores : text_fit, style, position, collision, readability, preservation).
<code>pagereconstruct/placement_solver.py</code>	Choisit le meilleur candidat sans collision , avant le rendu (normal → shrink → interligne compact → shift).
<code>pagereconstruct/collision_detector.py</code>	Détecte les chevauchements sur la géométrie réellement rendue (texte/texte, texte/protégé).
<code>pagereconstruct/block_expansion_solver.py</code>	Étend la bbox d'un bloc + reflow des voisins après traduction (le texte traduit est souvent plus long).
<code>pagereconstruct/multiblock_layout_solver.py</code>	Résout les collisions de page dense en optimisant des groupes de blocs dans les régions de flux.

Fichier	Rôle
<code>pagereconstruct/layout_reflow_solver.py</code>	Solveur d'espacement/reflow conservateur (CPU-only) qui préserve les garanties acquises.

Rendu : ops, dispatch, renderers, backends

Fichier	Rôle
<code>pagereconstruct/ops.py</code>	Opérations de rendu explicites — le backend les exécute, il n'improvise pas.
<code>pagereconstruct/render_ops.py</code>	RenderOps : instructions de dessin plates et résolues (dispatch + mesure ont lieu UNE fois ici, en espace PT).
<code>pagereconstruct/renderer_dispatcher.py</code>	Mappe rôle → renderer concret. Un rôle inconnu ne devient JAMAIS le renderer paragraphe.
<code>pagereconstruct/text_measure.py</code>	Mesure de texte canonique (largeur de ligne, wrap, ajustement) — une seule porte de mesure. Pure, pas de dessin.
<code>pagereconstruct/render_backend.py</code>	Backend raster (PIL) « style-aware » : efface les zones source via patches, puis dessine le texte traduit.
<code>pagereconstruct/backends/pdf_vector.py</code>	Backend PDF vectoriel (PyMuPDF) — sortie finale V1 . Fond + rectangles patch + texte via <code>insert_textbox</code> .
<code>pagereconstruct/backends/raster_debug.py</code>	Backend PNG de debug (overlays, contact sheets) — pas la sortie WYSIWYG.
<code>pagereconstruct/renderers/base.py</code>	Layout/mesure partagé + renderer de base (mesure d'abord → peint depuis la mesure).
<code>pagereconstruct/renderers/paragraph.py</code>	Paragaphes + items de liste.
<code>pagereconstruct/renderers/heading.py</code>	Titres.

Fichier	Rôle
<code>pagereconstruct/renderers/code.py</code>	Code.
<code>pagereconstruct/renderers/formula.py</code>	Formules.
<code>pagereconstruct/renderers/index.py</code>	Index.
<code>pagereconstruct/renderers/bibliography.py</code>	Bibliographie.
<code>pagereconstruct/renderers/table_cell.py</code>	Cellules de table.
<code>pagereconstruct/renderers/anchored_label.py</code>	Labels ancrés (+ variante review pour rôle inconnu).
<code>pagereconstruct/renderers/preservation.py</code>	Objets préservés (pixels).

Préservation, protection, traçabilité

Fichier	Rôle
<code>pagereconstruct/overlay_manager.py</code>	Gère underlays (sous le texte : figures, fonds, formules) vs overlays (au-dessus : numéros de page, labels), attribue le <code>z_index</code> .
<code>pagereconstruct/protected_region_index.py</code>	Index spatial des régions protégées (zones où le renderer n'écrit pas et le patch n'efface pas).
<code>pagereconstruct/text_removal_ledger.py</code>	Registre de suppression : une ligne par texte source remplacé — la suppression était-elle attendue et vérifiée. Garantit qu'aucun texte source ne fuit.
<code>pagereconstruct/source_text_lifecycle_ledger.py</code>	Cycle de vie par unité (décision traduction → entrée reconstruction → op de rendu → statut visuel).
<code>pagereconstruct/source_text_leak_detector.py</code>	 Détecte le texte source résiduel : une zone de patch quasi identique source/reconstruit = ancien texte non effacé (= leak).

Validation & QA

Fichier	Rôle
<code>pagereconstruct/validator.py</code>	Les findings gouvernent le statut `ok`
<code>pagereconstruct/quality.py</code>	Métriques de qualité dérivées du <code>PageRenderPlan</code> .
<code>pagereconstruct/visual_qa.py</code>	Score sur les 6 critères publication (présence texte/non-texte, overlap, position, typo).
<code>pagereconstruct/invariant_guard.py</code>	Verrouille les invariants à ne jamais régresser (toute ligne visible a un owner ; fond propre vérifié ; pas de fallback silencieux).
<code>pagereconstruct/style_similarity.py</code>	Score de fidélité du style résolu vs source (publication ≥ 0.95 ; < 0.85 = ko).

Auto-correction & ponts legacy

Fichier	Rôle
<code>pagereconstruct/autocorrect/correction_loop.py</code>	La validation corrige, pas seulement constate : compile → audit → corrige (knob) → recompile → garde le meilleur score.
<code>pagereconstruct/autocorrect/correction_plan.py</code>	Traduit les findings d'audit en actions de correction.
<code>pagereconstruct/autocorrect/retry_policy.py</code>	Politique de réessai de la boucle.
<code>pagereconstruct/legacy_contract_bridge.py</code>	Rend le savoir de l'ancien moteur (process_page / reconstructor) disponible dans le contrat moderne.
<code>pagereconstruct/legacy/function_registry.py</code>	Registre de migration des fonctions legacy (KEEP_AS_IS / ADAPT / WRAP / MERGE / DROP / REPLACE_TESTED).

Fichier	Rôle
<code>pagereconstruct/integration_adapter.py</code>	Adaptateur optionnel pour 2 modules externes « 95%-unlock ».
<code>pagereconstruct/ocr_typography_engine.py</code>	Améliore la typographie OCR (estimation font-em stable, échelle de style).
<code>pagereconstruct/templates/book_figure_page.py</code>	Template pour page de livre avec figure + légende.
<code>pagereconstruct/errors.py</code>	Exceptions du module.

6. `pubready/` — QA publication-ready (granulaire et explicable)

Hiérarchie d'audit : **page** → **blocs** → **phrases** → **dimensions**, chaque dimension comparée à l'origine PAGEPRINT. Une page ko / texte manquant / leak / objet détruit **bloque le document**.

Fichier	Rôle
<code>pubready/evaluator.py</code>	Entrée unique : normalise l'input traduit puis lance <code>evaluate_page</code> (additif, ne touche pas le rendu).
<code>pubready/page_auditor.py</code>	Évaluateur de page : lance les audits granulaires + réutilise la QA visuelle, combine selon les gates.
<code>pubready/document_auditor.py</code>	Consolidation document (score ≠ moyenne simple).
<code>pubready/gates.py</code>	Seuils stricts par étape + hard blockers + règles de décision page/document.
<code>pubready/schema.py</code>	Schémas de score granulaires/explicables.
<code>pubready/evidence.py</code>	Accès aux données d'origine PAGEPRINT pour comparer reconstruit vs source.
<code>pubready/reports.py</code>	Export des rapports (JSON + Markdown).
<code>pubready/stages/typography_audit.py</code>	Audit typo granulaire (classe police, taille, couleur, gras, italique, alignement).
<code>pubready/stages/translation_audit.py</code>	Audit traduction par bloc (présence, non-vide, troncature, tokens protégés, code/formule non traduits).
<code>pubready/stages/position_audit.py</code>	Audit position : la bbox de rendu reste ancrée sur la source, sans dérive ni collision.
<code>pubready/stages/background_audit.py</code>	Audit fond propre (trame nettoyée + registre complet + patches non destructeurs).
<code>pubready/stages/preservation_audit.py</code>	Audit préservation (formule/code/image/table/logo protégés et non écrasés).
<code>pubready/stages/intrablock_audit.py</code>	Audit composition intra-bloc (aucun mot perdu, pas d'overflow/clipping).
<code>pubready/stages/contract_audit.py</code>	Audit du <code>FinalReconstructionContract</code> (chaque bloc a layout/style/renderer).

Fichier	Rôle
<code>pubready/stages/render_ops_audit.py</code>	Audit RenderOps (BackgroundOp présent, TextOps couvrants, parité PDF/PNG).
<code>pubready/stages/pageprint_audit.py</code>	Audit PAGEPRINT (fournit-il assez pour reconstruire ?).
<code>pubready/stages/visual_image_audit.py</code>	● Regarde réellement l'image finale (cv2) : zones protégées ~identiques (sinon objet détruit), zones de texte remplacé ayant changé (sinon leak = ancien texte visible).

7. `translation_engines/` — moteurs de traduction enfichables

Fichier	Rôle
<code>__init__.py</code>	Moteurs pour les essais contrôlés PAGEPRINT/PAGETRANSLATE.
<code>base.py</code>	<code>TranslationEngine</code> + <code>TranslationResult</code> (interface commune).
<code>request.py</code>	<code>TranslationRequest</code> (DTO d'entrée).
<code>factory.py</code>	<code>create_translation_engine</code> (fabrique).
<code>model_registry.py</code>	Registre de modèles (sélection NLLB / M2M100 / Marian / Opus selon la paire de langues).
<code>ct2_engine.py</code>	Moteur CTranslate2 (encode source, préfixe cible, décode hypothèses).
<code>local_model_engine.py</code>	Moteur de modèle local.
<code>external_model_engine.py</code>	Moteur de modèle externe (API).
<code>rule_engine.py</code>	Moteur à règles.
<code>mock_engine.py</code>	Moteurs factices (Mock/Prefix/Echo) pour tests.
<code>translation_memory.py</code>	Mémoire de traduction validée (un hit fait sauter le modèle).
<code>placeholder_policy.py</code>	Choix du style de placeholder + construction/variantes.

Fichier	Rôle
<code>profile_store.py</code>	Charge <code>translation_profiles.json</code> / <code>style_tone_profiles.json</code> (seuils qualité, post-édition, glossaire, ton).
<code>engine_health.py</code>	Rapport de santé du moteur.

8. `pipeline_agents/` — agents LLM open-source par étape (P1-P7)

Chaque agent encapsule un LLM local (Phi-3.5 Mini, Qwen2.5, GGUF...) derrière une interface identique, provider-agnostique. Chacun a un équivalent heuristique sans LLM.

Fichier	Rôle
<code>__init__.py</code> / <code>base.py</code>	Paquet + classes de base (interface commune indépendante du backend).
<code>registry.py</code>	Factory <code>get_agent(...)</code> provider-agnostique.
<code>heuristics.py</code>	Estimateurs déterministes (sans LLM) pour P1, P3, P5, P6.
<code>p1_extraction.py</code>	P1 — extraction & segmentation sémantique (titres, formules, légendes, frontières).
<code>p2_structure.py</code>	P2 — structuration hiérarchique (parent/enfant, sections, regroupement).
<code>p3_layout.py</code>	P3 — mode de layout (<code>inline_reflow</code> / <code>preserve_line_breaks</code> / <code>preserve_paragraphs</code>).
<code>p4_translation.py</code>	P4 — validation & post-édition de traduction (score, problèmes, version corrigée).
<code>p4_qe_estimator.py</code>	P4 QE — estimation qualité sans génération LLM (heuristique / CometKiwi...).
<code>p5_render.py</code>	P5 — stratégie de rendu (<code>prose_reflow</code> / <code>label_stack...</code>).
<code>p6_background.py</code>	P6 — audit du fond (artéfacts textuels résiduels, zones à re-painter).
<code>p7_publication_layout.py</code>	P7 — mise en page finale publication-ready au niveau page.

9. `server/` — couche HTTP mince (clone moderne)

Fichier	Rôle
<code>server/__init__.py</code>	Le serveur ne contient plus l'intelligence : upload, routing, appel du pipeline, retour JSON.
<code>server/api.py</code> (3911.)	API du clone : upload multi-fichiers, routing, appel de l'orchestrateur, persistance des résultats.
<code>server/static/index.html</code>	Front statique minimal.

10. `tools/` — outils internes (audit, démos, patches, Rule Studio)

Audits & essais

Fichier	Rôle
<code>tools/audit_text_survival.py</code>	Garantit que 100% du texte PAGEPRINT se rendent (traduit/préservé/exclu avec raison).
<code>tools/audit_translation_selection.py</code>	Audit de la qualité de sélection de traduction choisie, ce qui est exclu/protégé.
<code>tools/run_functional_audit.py</code> / <code>run_batch_functional_audit.py</code>	Audit fonctionnel strict (rev_04) sur l' <code>INPUT</code> unitaire ou en lot.
<code>tools/run_translation_trial.py</code> / <code>run_batch_translation_trial.py</code>	Essais de traduction contrôlés, unitaire ou en lot.
<code>tools/run_document_trial.py</code>	Extrait un PDF puis lance un essai de traduction.
<code>tools/run_pageprint_pagetranslate_audit.py</code>	Audit PAGEPRINT→PAGETRANSLATE sur des exemples aléatoires.
<code>tools/run_pipeline_full_demo.py</code>	Pipeline complet (PAGEPRINT→PAGETRANSLATE→PAGER sur des pages, dump des artefacts).
<code>tools/compare_legacy_reconstruction.py</code>	Compare ancien/nouveau rendu + score puis (heatmap de diff).
<code>tools/check_translation_engine.py</code> / <code>translate_text_smoke.py</code>	Smoke-tests moteur/placeholders.

Fichier	Rôle
<code>test_placeholder_roundtrip.py</code>	
<code>tools/export_pipeline_inventory.xlsx.py</code>	Inventaire page par page en XLSX (un onglet par page)

Demos locales

Fichier	Rôle
<code>tools/local_demo_app.py</code>	App Tkinter locale (choisir PDF, pages, niveau de pipeline, lancer).
<code>tools/local_demo_runner.py</code>	Backend CLI de l'app ci-dessus.
<code>tools/demo_studio_backend.py</code>	Backend local (non-web) pour un « Demo Studio » Flutter (émet des évènements).

Patches ponctuels (`tools/patch_*.py`)

Scripts qui **patchent du code** de façon ciblée (correctifs déterministes appliqués à des fichiers précis) : `patch_atomic_patch_planner`, `patch_atomic_text_builder`, `patch_atomic_text_projection`, `patch_background_cleaner_deterministic`, `patch_demo_backend_background_translation`, `patch_demo_studio_main`, `patch_plan_compiler_spacing_reflow`, `patch_projection_consume_translations`, `patch_text_survival_builder/projection/reader`, `patch_vsense_studio_main`. Ce sont des correctifs d'historique (utiles pour comprendre les bugs déjà traités, dont certains touchent ton sujet : `spacing_reflow`, `background_cleaner`, `text_survival`).

`tools/pipeline_dashboard/` — tableau de bord Streamlit

Fichier	Rôle
<code>app.py</code>	Dashboard dense et éditable (1 ligne = 1 élément granulaire) pour vérifier l'application des règles page par page.
<code>elements.py</code>	Construit les enregistrements plats (jointure PAGEPRINT × PAGETRANSLATE).
<code>ingest.py</code>	Ingestion d'un run d'audit en base SQLite + copie des assets.

`tools/rule_studio/` — console de gouvernance des règles

Outil complet qui **scanne le dépôt, extrait les règles dispersées, les classe par étape de pipeline, les audite, les simule et propose des patches tracés** (jamais d'écriture silencieuse).
Sous-paquets :

- `app.py` / `cli.py` : interfaces Streamlit / ligne de commande.
- `core/` : `models` (`RuleRecord`), `scanner`, `classifier`, `rule_registry`, `simulator` (DSL sûr sans eval/exec), `usage_analyzer`, `patcher`, `git_guard`, `test_runner`, `exporters`, `studio` (façade), `pipeline_audit_bridge`.
- `extractors/` : `python_ast`, `config`, `markdown`, `comment_rule`, `schema`.
- `storage/` : `rule_store` (SQLite), `migrations`.
- `agents/` : agents IA (`model_client`, `rule_interpreter_agent`, `rule_coding_agent`, `rule_validation_agent`, `agent_runner`, `prompts`) — l'IA **explique et propose**, les tests **prouvent**, aucun agent n'applique de patch.

11. `scripts/` — scripts d'expérimentation / batch

Fichier	Rôle
<code>scripts/run_extraction_40pages.py</code>	Extraction complète des 40 premières pages de 3 PDF (dump hiérarchique).
<code>scripts/audit_p1_extraction.py</code>	Audit P1 sur de vrais PDF (couverture des champs).
<code>scripts/run_reconstruction.py</code>	Reconstruction depuis les données extraites (coordonnées en points, ordre dessins→images→texte).
<code>scripts/run_reconstruction_validation.py</code> (14491.)	Validation de reconstruction (texte attendu, tokens protégés...).
<code>scripts/run_reconstruction_validation_random20.py</code>	Idem sur 20 pages aléatoires.
<code>scripts/run_comparative_analysis.py</code>	Analyse comparative pixel à pixel (RMSE global et par zone 3×3).
<code>scripts/run_background_master_random_40.py</code>	Génère/échantillonne le background master sur 40 pages aléatoires.
<code>scripts/run_object_comprehension_random_audit.py</code>	Audit de compréhension d'objets sur pages aléatoires.
<code>scripts/run_final_page_compiler_random40.py</code>	Lance le compilateur de page final sur 40 cas.

Fichier	Rôle
scripts/run_pageprint_audit.py	Audit PAGEPRINT sur pages réelles.
scripts/run_perfect_reconstruction_experiment.py	Expérience de reconstruction via l'extracteur « parfait ».
scripts/visualize_perfect_extraction_random40.py	Rend des PNG annotés (bbox texte/image/vecteur/table/formule
scripts/metadata_explorer_builder.py	Construit un explorateur de métadonnées.
scripts/download_visual_layout_models.py	Télécharge les modèles de layout visuel.

12. tests/ — ~125 tests (le contrat exécutable du projet)

Les tests **encodent les règles** que le pipeline ne doit pas casser. Par famille :

Dossier	Nb	Ce qu'ils vérifient
tests/pageprint/	31	Détection (TOC, index, table, légende, figure, en-tête/pied répété, watermark, marque éditeur), préservation des numéros de page/figure, plans de traduction (pas de ligne brute d'index/table/caption traduite, pas de rôle <code>None</code> , pas de token <code>word/char</code>), santé des titres.
tests/pagereconstruct/	28	● Backends, résolveur de fond, séparation fond/source, expansion de bloc, détecteur de collision, contrat final, fuite de texte source , registre de suppression, survie du texte , pas de double rendu, invariant d'ordre de lecture, dispatcher, similarité de style, gate de publication. À exécuter en premier pour ton debug.
tests/pagetranslate/	22	Couverture de traduction, dédup parent/enfant, verrous de glossaire, préservation des tokens techniques/MLP-CNN, roundtrip des placeholders, projection (bboxes, rôles, tous originaux préservés), fallback batch, raisons de QA.
tests/pubready/	10	Cœur pubready, stages, préservation, intra-bloc, typo (M7), modes M8-M11, audit du cycle de vie du texte source, blockers d'audit visuel d'image.
tests/translation_engines/	13	Batching/décodage CT2, préfixes de langue M2M100/Marian, fallbacks du registre de modèles, mémoire de traduction (hit exact / ignore non validé), variantes de placeholder ASCII/XML, store de profils.
tests/integration/	4	Cas réels de reconstruction, absence d'overlay source (pubready), gates de publication, cas DocIntelligence.
tests/functional/	7	Audit batch (plan vide, rôles manquants, pas de fallback), validateur strict PAGEPRINT, runner d'essai de traduction.
tests/pipelines/	2	Routeur OCR (image-dominant, texte natif insuffisant).
tests/legacy_reconstruction/	1	Contrats legacy.

<code>tests/</code> (racine, 25)	— Tests des modules racine : <code>coverage_validator</code> , <code>pageprint</code> , <code>pagetranslate</code> , <code>pipeline_agents</code> , <code>pipeline_orchestrator</code> , <code>reconstructor_font_sizing</code> ● (taille de police — lié à ton bug), <code>background_inpainter</code> , <code>layout_descriptor</code> (v3), <code>llm_semantic_corrector</code> , <code>positioning_policy</code> , <code>publication_qa</code> , etc.
----------------------------------	--

En résumé pour ton debug `pagereconstruct/`

Les deux pannes que tu décris (texte source non effacé + bbox qui écrase tout sur la 1^{re} ligne) se concentrent dans une poignée de fichiers :

1. `layout_box_resolver.py` — le bug « texte redessiné dans la bbox de la 1^{re} ligne » est exactement ce qu'il est censé corriger (`layout_bbox` = bloc complet vs `anchor_bbox` = 1^{re} ligne).
2. `patch_planner.py` + `background_resolver.py` + `background_contract.py` — l'effacement du texte source (patches / fond propre).
3. `source_text_leak_detector.py` et `pubready/stages/visual_image_audit.py` — la détection que l'effacement a échoué.
4. `font_size_sanitizer.py` + le test `test_reconstructor_font_sizing.py` — la taille de police effondrée (texte illisible).

Tests à lancer en priorité : `tests/pagereconstruct/test_source_text_leak.py`,
`test_text_survival.py`, `test_layout_and_typography.py`, et
`tests/test_reconstructor_font_sizing.py`.